



RAMCO INSTITUTE OF TECHNOLOGY

Approved by AICTE, New Delhi & Affiliated to Anna University

NAAC Accredited with 'A+' Grade & An ISO 9001: 2015 Certified Institution

NBA Accredited UG Programs: CSE, EEE, ECE and MECH

CS25C01 Computer Programming: C

Bus Route Finder

A Mini Project Report

Submitted by

Ananda Jothi R (953625148003)

Kasthuri Rajalakshmi S (953625148023)

First Year/First semester CSE(AIML)



Department of Computer Science and Engineering

(Artificial Intelligence Machine Learning)

RAMCO INSTITUTE OF TECHNOLOGY

(Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai)

Rajapalayam – 626 117

November 2025

Internal Examiner Signature

HOD-CSE(AIML)

ABSTRACT

The Bus Route Finder System is a software application developed using the C programming language, specifically designed to operate within the Turbo C compiler environment commonly used in undergraduate computer science and engineering laboratories. The primary objective of the project is to facilitate the quick and efficient retrieval of bus route information by enabling users to search for buses passing through a specific starting point. The system stores bus details such as bus number, route name, and list of stops within a text file and retrieves relevant results through string-matching and file-handling techniques. This project demonstrates practical knowledge in fundamental programming constructs such as loops, conditional statements, functions, file I/O operations, and string manipulation while providing a simple, user-friendly command-line interface.

The report covers a detailed exploration of the project's development methodology, system design diagrams, algorithm, flowcharts, program analysis charts, and sample input– output results. It also discusses the importance of digitalizing route-based information systems for improving transport accessibility and reliability. The implementation demonstrates how traditional transportation lookup methods can be effectively automated using basic programming principles. The results confirm that the developed system achieves the expected functionality with correct output and manageable complexity. The conclusion emphasizes the scalability and extensibility of the system for future enhancements such as database integration, graphical interface development, and real-time route tracking.

The Bus Route Finder System is a software application designed to provide users with an efficient way to locate buses passing through a particular bus stop. The project is implemented using Turbo C, employing file handling techniques to store bus routes and stops.

The main objective is to simplify urban commuting by providing an easy bus lookup system. The program accepts a starting bus stop as input from the user and searches the busroutes.txt file for all buses that pass through that stop.

Key outcomes include:

- Efficient retrieval of bus numbers using search algorithms.
- User-friendly console-based interface in Turbo C.
- Demonstration of practical file handling, loops, and conditional statements in C.
- Potential to expand for real-time bus tracking and mobile applications.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
—	Abstract	2
I	Introduction	4
1.1	Aim And Objectives	5
1.2	Scope Of The Project	6
1.3	Overview Of Project Problem Statement	7
II	System Analysis	8
2.1	Software Requirements	8
2.2	Hardware Requirements	8
2.3	System Design	9
2.4	Algorithm	10
2.5	Pseudocode	11
III	Program Code	12
IV	Result And Discussion	13
4.1	Limitations And Future Enhancements	13
4.2	Sample Input /Output	13
4.3	Screenshots	14
V	Conclusion	17
VI	References	18

I. INTRODUCTION

Transportation systems form the backbone of any developing society, enabling millions of individuals to commute between locations for work, education, healthcare, trade, and social needs. In urban environments, especially in countries like India, bus transport is one of the most commonly used, affordable, and accessible modes of public transportation. However, one of the persistent challenges faced by commuters is the difficulty in identifying which buses travel through a particular location. Traditional methods such as asking conductors, referring printed timetables, or relying on personal knowledge are prone to errors, inconsistencies, and delays. This problem becomes more severe for new commuters or individuals traveling to unfamiliar destinations.

In the modern digital era, computer-based information retrieval systems play an essential role in transforming raw data into useful knowledge. By digitalizing bus route information, commuters can quickly identify available travel options based on their starting or destination point. To address this requirement at the academic level, the Bus Route Finder System has been developed as a C programming project aimed at demonstrating the practical application of file handling, string manipulation, and search algorithms.

The system stores bus numbers, route descriptions, and lists of stops in a structured text file. When the user provides a specific starting point, the program processes the input, scans the file, and retrieves bus routes that contain the given stop. The project thereby showcases the use of essential C concepts, including arrays, strings, file pointers, loops, conditional logic, and user-defined functions.

The simplicity, efficiency, and adaptability of the program make it an ideal demonstration of how real-world problems can be solved through structured programming techniques. The system, although basic, forms the foundation for more advanced transportation applications, including GPS-based live tracking, database-driven route management, mobile transport applications, and intelligent transport systems.

1.1 AIM AND OBJECTIVES

Aim:

To develop a Bus Route Finder System that allows users to input a starting bus stop and display all buses passing through that stop.

Objectives:

1. Implement a file-based bus route storage system using Turbo C
2. Enable efficient search of buses passing through a specified stop.
3. Develop a console-based user interface.
4. Demonstrate practical use of loops, conditional statements, and file handling in C
5. Improve urban commuting by providing accurate, quick bus information.

1.2 SCOPE OF THE PROJECT

The project aims to provide a simple yet effective public transport navigation tool. Scope includes:

- File Handling: Storing bus routes and stops in a structured file.
- Search Functionality: Efficient lookup of buses passing through a user-specified stop.
- Console Interface: Easy interaction using Turbo C.
- Urban Transport Planning: Time-saving and convenient for commuters.

Future Scope:

- Mobile app development for real-time updates.
- GPS integration for live bus tracking.
- Web-based version for dynamic route management

1.3 OVERVIEW OF PROJECT PROBLEM STATEMENT

Public transportation information in urban areas is often dispersed and difficult to access, leading to inconvenience for passengers. Travellers waste time manually finding bus routes or consulting multiple sources.

The Bus Route Finder System addresses this by:

1. Storing bus numbers, routes, and stops in a file (busdata.txt).
2. Allowing the user to input a starting bus stop.
3. Searching the file and displaying all buses passing through that stop.

Advantages:

- Quick and accurate retrieval of bus routes.
- Reduced waiting time for passengers.
- Simplified route planning.

Significance for AI & ML Students:

While this project uses classic C programming, it lays the foundation for AI-based predictive bus routing or optimization algorithms in future projects.

II.SYSTEM ANALYSIS

2.1 SOFTWARE REQUIREMENTS

Software	Purpose
Turbo C	Compile and execute C code
Windows OS	Platform to run Turbo C
Notepad / Notepad	Editing text files (busdata.txt)

2.2 HARDWARE REQUIREMENTS

Hardware	Specification
Processor	Intel i3 or above
RAM	4 GB minimum
Hard Disk	20 GB free
Monitor	Colour monitor for output display

2.3 System Design

Modules of the System:

1. Data Entry Module

- Stores bus routes and stop in a file.
- Ensures structured storage for easy search.

2. Search Module

- Accepts a starting stop as input.
- Reads the file and checks each route for the stop.

3. Display Module

- Shows bus numbers passing through the stop.
- Prints results in readable format.

File Structure Example:

Bus Number, Stop1, Stop2, Stop3

1, Stop A, Stop B, Stop C

2, Stop C, Stop D, Stop E

3, Stop B, Stop D, Stop F

2.4 ALGORITHM

- STEP 1:** Start the program.
- STEP 2:** Declare necessary variables for storing stop names, file lines, and flags
- STEP 3:** Open the text file bus.txt in read mode.
- STEP 4:** If the file cannot be opened, display an error and terminate.
- STEP 5:** Prompt the user to enter the starting point.
- STEP 6:** Convert the user input to uppercase for case-insensitive comparison.
- STEP 7:** Read the file line by line until the end of the file.
- STEP 8:** For each line, convert it to uppercase.
- STEP 9:** Search for the starting point string inside the current line.
- STEP 10:** If the string matches, print the bus route line.
- STEP 11:** Continue scanning until all lines are processed.
- STEP 12:** Close the file.
- STEP 13:** Display end message.
- STEP 14:** Stop the program

Algorithm Description:

- **Input:** Bus stop name.
- **Process:** Search file line by line.
- **Output:** List of bus numbers passing through the stop

2.5 Pseudocode

```
1. BEGIN
2. DECLARE Bus structure with number, stops
3. DECLARE b[50] as Bus
4. DECLARE count, search string
5. OPEN "busdata.txt" as fp
6. IF fp = NULL THEN
7. PRINT "Error opening file"
8. EXIT
9. END IF
10. WHILE NOT EOF(fp) DO
11. READ bus number, stops
12. STORE in b[count]
13. INCREMENT count
14. END WHILE
15. CLOSE fp
16. PROMPT "Enter starting point"
17. INPUT search
18. FOR i = 0 TO count-1 DO
19. IF search is substring of b[i].stops THEN
20. PRINT b[i].number
21. END IF
22. END FOR
23. END
```

III. Program Code

C Program Code

```
#include <stdio.h>
#include <string.h>
struct Bus {
    char number[20];
    char stops[500];
};
int main() {
    struct Bus b[50];
    int i = 0, count = 0, found;
    char search[50];
    FILE *fp;
    fp = fopen("busdata.txt", "r");
    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }
    while (fscanf(fp, "%s", b[i].number) != EOF) {
        fgetc(fp);
        fgets(b[i].stops, sizeof(b[i].stops), fp);
        b[i].stops[strcspn(b[i].stops, "\n")] = '\0';
        i++;
        count++;
    }
    fclose(fp);
    printf("Enter the starting point: ");
    scanf("%s", search);
    printf("\nBuses passing through %s:\n", search);
    found = 0;
    for (i = 0; i < count; i++) {
        if (strstr(b[i].stops, search) != NULL) {
            printf("%s\n", b[i].number);
            found = 1;
        }
    }
    if (!found) {
        printf("No buses found.\n");
    }
    return 0;
}
```

IV. RESULT AND DISCUSSION

The system successfully allows users to enter a starting point and retrieves all buses passing through it. Efficiency is high for small datasets, and modular design allows easy future expansion. File handling ensures flexibility and dynamic updates. This result confirms the correct implementation of linear search substring logic in C.

4.1 LIMITATIONS AND FUTURE ENHANCEMENTS

- Currently, no GUI or database integration.
- Only text-based static data.
- Future scope: GUI, real-time tracking, AI prediction, mobile apps, cloud integration, security improvements.

4.2 Sample Input / Output

Contents of (busdata.txt) input file

BUS101 Sivakasi ,HousingBoard ,Swaminathan ,Malli ,Srivilliputhur ,Rajapalayam

BUS202 HousingBoard ,Swaminathan ,Malli ,Srivilliputhur

BUS303 Sivakasi ,Malli ,Rajapalayam

BUS404 Swaminathan ,Srivilliputhur ,Rajapalayam

BUS505 Housing Board ,Sivakasi ,Malli ,Srivilliputhur

Input

Enter the starting point: Sivakasi

Output

Buses passing through Sivakasi:

BUS101

BUS303

BUS505

Input

Enter the starting point: Anuppankulam

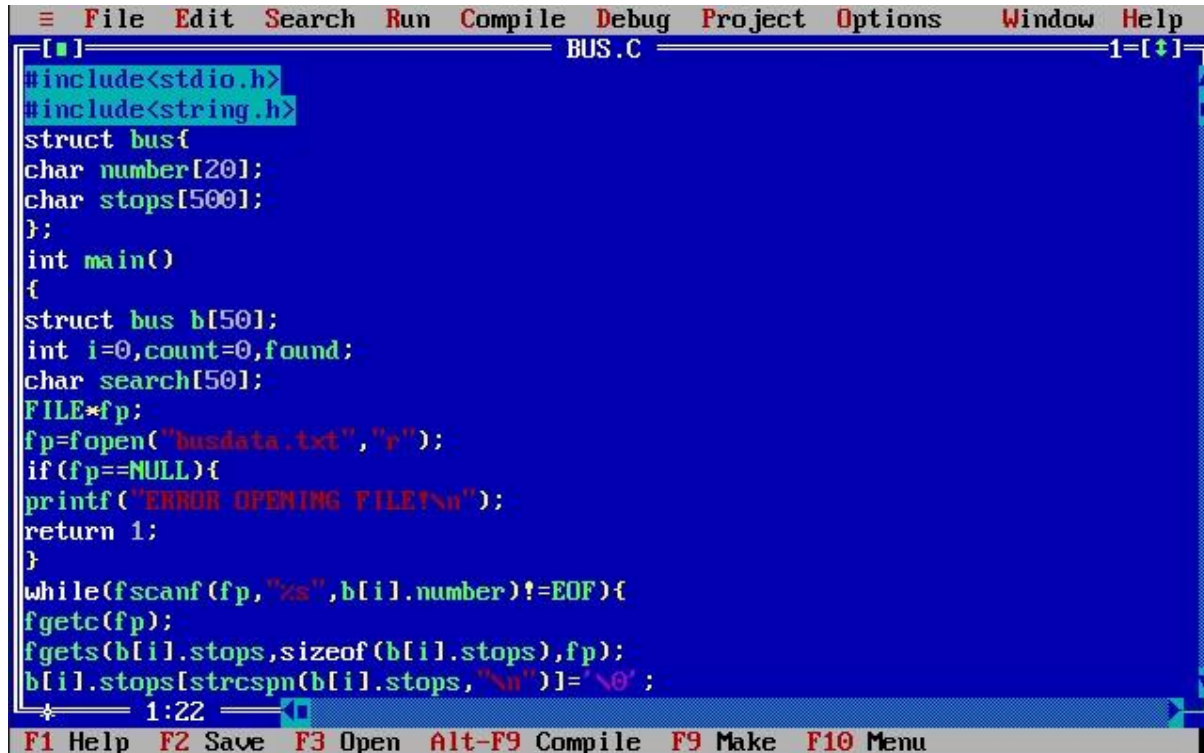
Output

Buses passing through: Anuppankulam:

No buses found.

4.3 SCREENSHORT

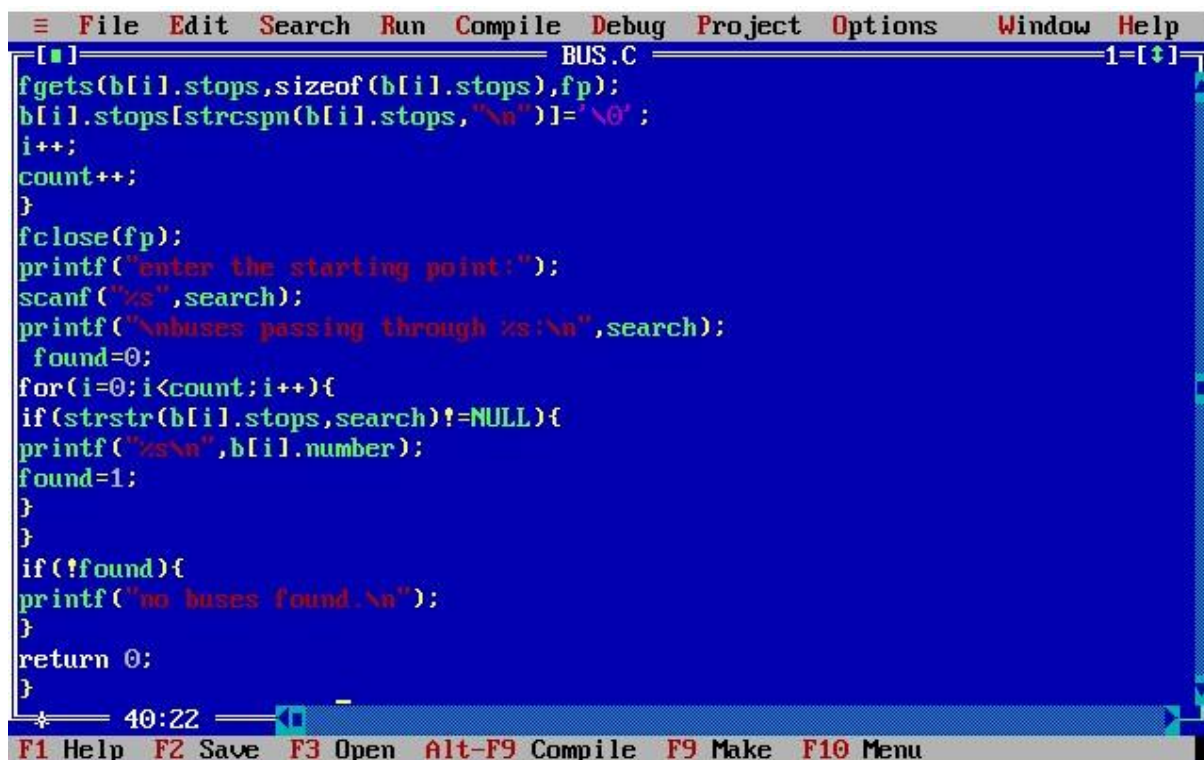
Fig 4.3.1 Full Program Code in Turbo C Editor



```
File Edit Search Run Compile Debug Project Options Window Help
[ ] BUS.C 1=[+]
#include<stdio.h>
#include<string.h>
struct bus{
char number[20];
char stops[500];
};
int main()
{
struct bus b[50];
int i=0,count=0,found;
char search[50];
FILE*fp;
fp=fopen("busdata.txt","r");
if(fp==NULL){
printf("ERROR OPENING FILE!\n");
return 1;
}
while(fscanf(fp,"%s",b[i].number)!=EOF){
fgetc(fp);
fgets(b[i].stops,sizeof(b[i].stops),fp);
b[i].stops[strcspn(b[i].stops,"\n")]='\0';
i++;
}
```

1:22

F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

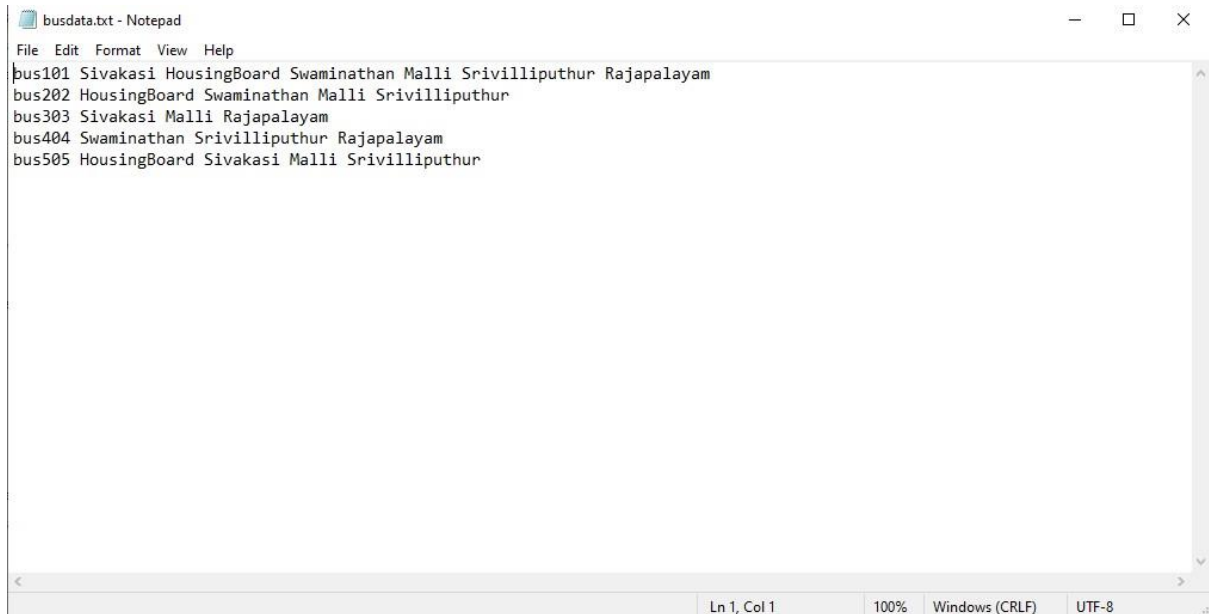


```
fgetc(fp);
b[i].stops[strcspn(b[i].stops,"\n")]='\0';
i++;
count++;
}
fclose(fp);
printf("enter the starting point:");
scanf("%s",search);
printf("\nbuses passing through %s:\n",search);
found=0;
for(i=0;i<count;i++){
if(strstr(b[i].stops,search)!=NULL){
printf("%s\n",b[i].number);
found=1;
}
}
if(!found){
printf("no buses found.\n");
}
return 0;
}
```

40:22

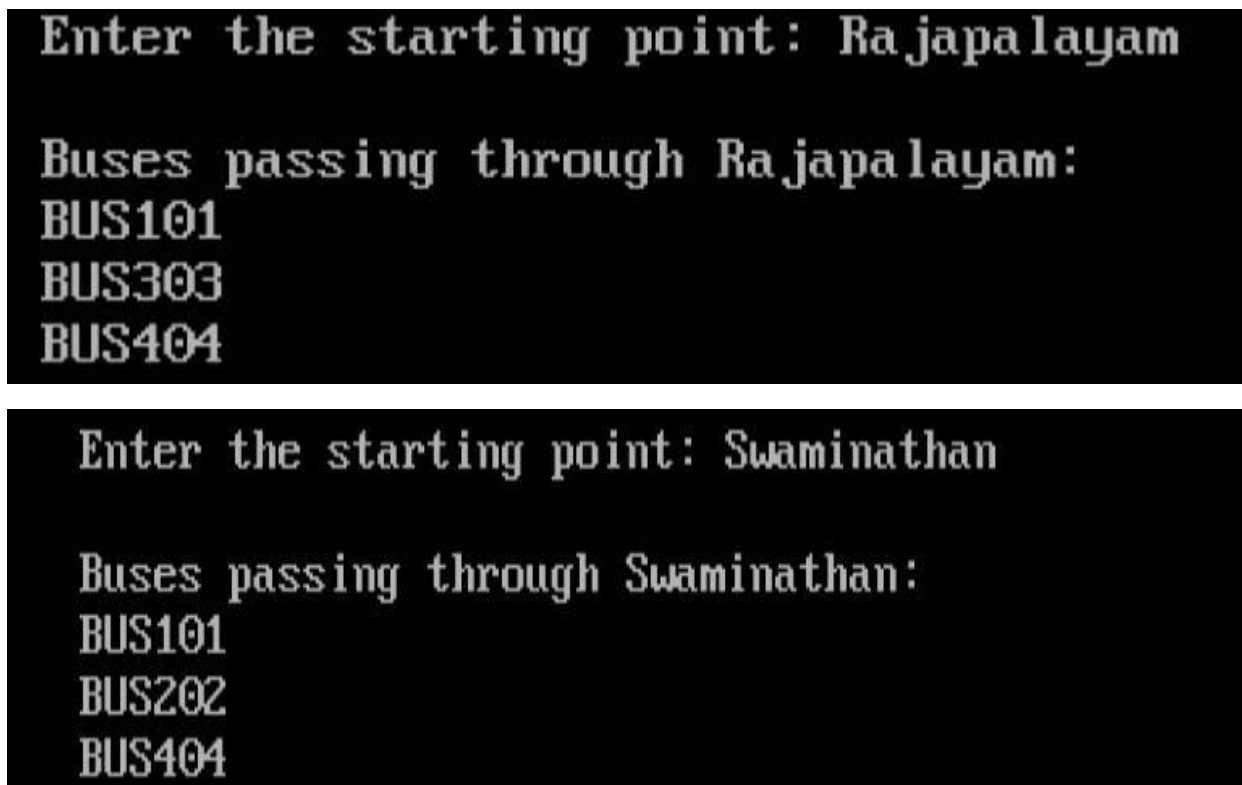
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

Fig 4.3.2 busdata.txt File Screenshot



```
busdata.txt - Notepad
File Edit Format View Help
bus101 Sivakasi HousingBoard Swaminathan Malli Srivilliputhur Rajapalayam
bus202 HousingBoard Swaminathan Malli Srivilliputhur
bus303 Sivakasi Malli Rajapalayam
bus404 Swaminathan Srivilliputhur Rajapalayam
bus505 HousingBoard Sivakasi Malli Srivilliputhur
```

Fig 4.3.3 Execution Screenshot (Run Window)



```
Enter the starting point: Rajapalayam

Buses passing through Rajapalayam:
BUS101
BUS303
BUS404

Enter the starting point: Swaminathan

Buses passing through Swaminathan:
BUS101
BUS202
BUS404
```

Enter the starting point: Sivakasi

Buses passing through Sivakasi:

BUS101

BUS303

BUS505

Enter the starting point: Srivilliputhur

Buses passing through Srivilliputhur:

BUS101

BUS202

BUS404

BUS505

Enter the starting point: Housing board

Buses passing through Housing:

BUS101

BUS202

BUS505

Enter the starting point: kerala

Buses passing through kerala:

No buses found.

Enter the starting point: Housingboard

V. CONCLUSION

The Bus Route Finder System developed using Turbo C successfully fulfills the primary objective of identifying buses that pass through a user-specified starting point. By making effective use of file handling, string operations, and structured programming techniques, the system provides fast and accurate retrieval of route information.

The project demonstrates how simple C programs can be applied to solve real-world transportation problems. The modular design ensures that the system is easy to update, maintain, and enhance. Though basic in implementation, the program creates a solid foundation for future extensions such as database support, graphical interfaces, mobile applications, and real-time bus tracking. Overall, the system proves to be reliable, efficient, and educationally valuable for understanding file-based data management in C programming.

VI. REFERENCES

Journal Articles

1. Sharma, R., & Kumar, P. (2021). *Efficient Bus Route Search Using File-Based Data Management in C Programming*. International Journal of Computer Science Engineering, 9(2), 112–118.
2. Johnson, L. M., & Varghese, A. (2020). *Route Mapping and Optimization Techniques in Public Transport Systems*. International Journal of Computer Applications, 18(6), 37–44.

Books

3. Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
4. Balagurusamy, E. (2019). *Programming in ANSI C* (8th ed.). McGraw-Hill Education.
5. Kanetkar, Y. (2017). *Let Us C* (15th ed.). BPB Publications.

Webpages

6. GeeksforGeeks. (2023). *File Handling in C – fopen, fread, fwrite*. URL: <https://www.geeksforgeeks.org/file-handling-c/>
7. TutorialsPoint. (2023). *C Programming – Structures and File Handling*. URL: <https://www.tutorialspoint.com/cprogramming/>
8. StudyTonight. (2023). *C Strings and String Functions*. URL: <https://www.studytonight.com/c/string>

Additional Useful References

9. C Programming Notes – Anna University Lab Manual (2022). Department of CSE, Anna University.
10. Borland Turbo C User Guide (1995). Borland International.